Developer Performance Analytics & Ranking System

Scenario

You are working as a Python Developer at a software company.
The management wants a **console-based system** to analyze developer performance and introduce a **gamified ranking system** to identify top performers.

The system should analyze:

- Task completion
- Code quality scores
- Team contributions
- Developer activity

Additionally, the system should include a **Developer Ranking Game** where developers compete based on performance.

Instructions

- Use **Python only (no external libraries)**
- Work in **Jupyter Notebook / Google Colab**
- Write **clean, readable, well-commented code**
- Handle **invalid inputs gracefully**
- Ensure the program **does not crash**
- Use **functions, loops, and proper structure**
- Submit a **.ipynb file**

Sample Dataset (Minimum Required)

Students must use and extend this dataset (add at least 10 developers):

```
developers = [
  {"id": 301, "name": "Ali", "team": "Backend", "tasks": 6, "score": 78, "year": 2022},
  {"id": 302, "name": "Sara", "team": "Frontend", "tasks": 8, "score": 88, "year": 2021},
  {"id": 303, "name": "Usman", "team": "DevOps", "tasks": 2, "score": 60, "year": 2023},
]
```

Part A: Python Foundations (5 Marks)

1. Display a welcome message
2. Store and display total number of developers
3. Ask user to input a team name
4. Display average code quality score
5. Check if active developers > inactive developers
   (Active = tasks > 5)

Part B: Data Structures & Control Flow (5 Marks)

6. Store unique teams using a collection
7. Display first 5 developer names
8. Create dictionary (developer ID : score)
9. Categorize developers:
   - Tasks < 3 as Low Productivity
   - 3–7 as Moderate
   - 7 as High Performer
10. Count developers above performance benchmark

Part C: Functions & Logic (5 Marks)

11. Function for average score per team
12. Function for top-performing developer

13. Function for developers by joining year
14. Function for performance category:

- < 50 : Poor
- 50–75 : Average
- 75 : Excellent

15. Nested loop report:

- Developers per team per category

Part D: Error Handling (5 Marks)

16. Safely accept developer ID input
17. Handle case where no developer found by year
18. Function:

- Return developer if exists
- Raise error if not

19. Validate input:

- Reject non-numeric IDs
- Show message if developer not found

20. Always print:
    "Developer lookup process completed"

Part E: Developer Ranking Game (15 Marks)

Performance Formula:

Final Score = (tasks × 5) + code_quality_score

Tasks:

21. Calculate final score for each developer
22. Display leaderboard (highest to lowest)
23. Declare best-performing developer

24. Head-to-head battle:

- Compare two developers
- Display winner

25. Assign levels:

- < 80 : Bronze
- 80–120 : Silver
- 120 : Gold

Submission Requirements

- Jupyter Notebook (.ipynb)
- Proper comments and structure
- Clean output
- No runtime errors